Parte 3. Arquitectura

Guía de estudio del sistema Triverde · Grupo 26 · Ingeniería de Software

Contenido

- 1. Las tres capas
- 2. Tecnologías y por qué
- 3. El servidor (Controladores y Persistencia)
- 4. Las Vistas (PWA)
- 5. Operación sin conexión
- 6. El recorrido completo
- 7. Instalación y despliegue
- 8. Glosario
- 9. Preguntas de repaso

Esta parte explica cómo está construido el sistema por dentro: las capas, las tecnologías y por qué se eligieron, cómo se organiza el código, qué piezas comparten todos los módulos, cómo funciona sin conexión y cómo se instala. Los archivos citados están en el repositorio del grupo (`backend/` y `frontend/`). En los entregables la capa de presentación se llama **Vistas**, no "frontend": así lo pide el profesor.

1. Las tres capas

El sistema es una aplicación web en tres capas, que la asignatura nombra **Vistas / Controladores / Persistencia**:

```
VISTAS                      CONTROLADORES                    PERSISTENCIA
React + Vite (PWA)  --->    Django REST Framework   --->     PostgreSQL
en el navegador      JSON   en el servidor          ORM      base relacional
                    <---                            <---
```

Capa	Dónde vive	Qué hace	Qué NO hace
Vistas (`V_`)	`frontend/src/pages/`	Muestra datos, captura lo que ingresa el usuario y lo envía. Guarda la cola sin señal	No calcula nada del negocio: ni peso, ni mezcla, ni tarifas
Controladores (`C_`)	`backend/<app>/views.py` y `services.py`	Recibe cada petición, revisa el permiso, valida, ejecuta la regla de negocio, audita y responde	No dibuja pantallas
Persistencia	`backend/<app>/models.py` → tablas en PostgreSQL	Guarda los datos y hace cumplir restricciones de integridad	No contiene la lógica de los casos de uso

Dentro del servidor, la regla de negocio de cada caso de uso vive en los **servicios** (`services.py`, y en algunos módulos archivos como `certificados.py`, `sinader.py` o `cumplimiento.py`). El controlador recibe, valida, llama al servicio y responde. Así el mismo cálculo sirve para cualquier pantalla o dispositivo.

Vistas y Controladores son **dos programas separados** que solo se hablan por HTTP con datos en JSON. Esa separación es la que permite que las Vistas sigan funcionando sin señal: guardan lo registrado y lo envían cuando vuelve la conexión.

1.1 Las vistas de arquitectura (modelo 4+1)

Vista 4+1	Qué muestra	Artefacto del proyecto
Escenarios	Qué hace el sistema para cada actor	Diagramas de casos de uso y CU extendidos (Parte 2)
Lógica	Las entidades y sus relaciones	Modelo de datos en cuatro niveles y diagrama de clases (Parte 4)
Procesos	Cómo colaboran las piezas en el tiempo	Diagramas de secuencia (Parte 6)
Desarrollo	Cómo se organiza el software	Diagrama de componentes (Figura 3.1)
Física	Dónde corre cada pieza	Diagrama de despliegue (Figura 3.2)

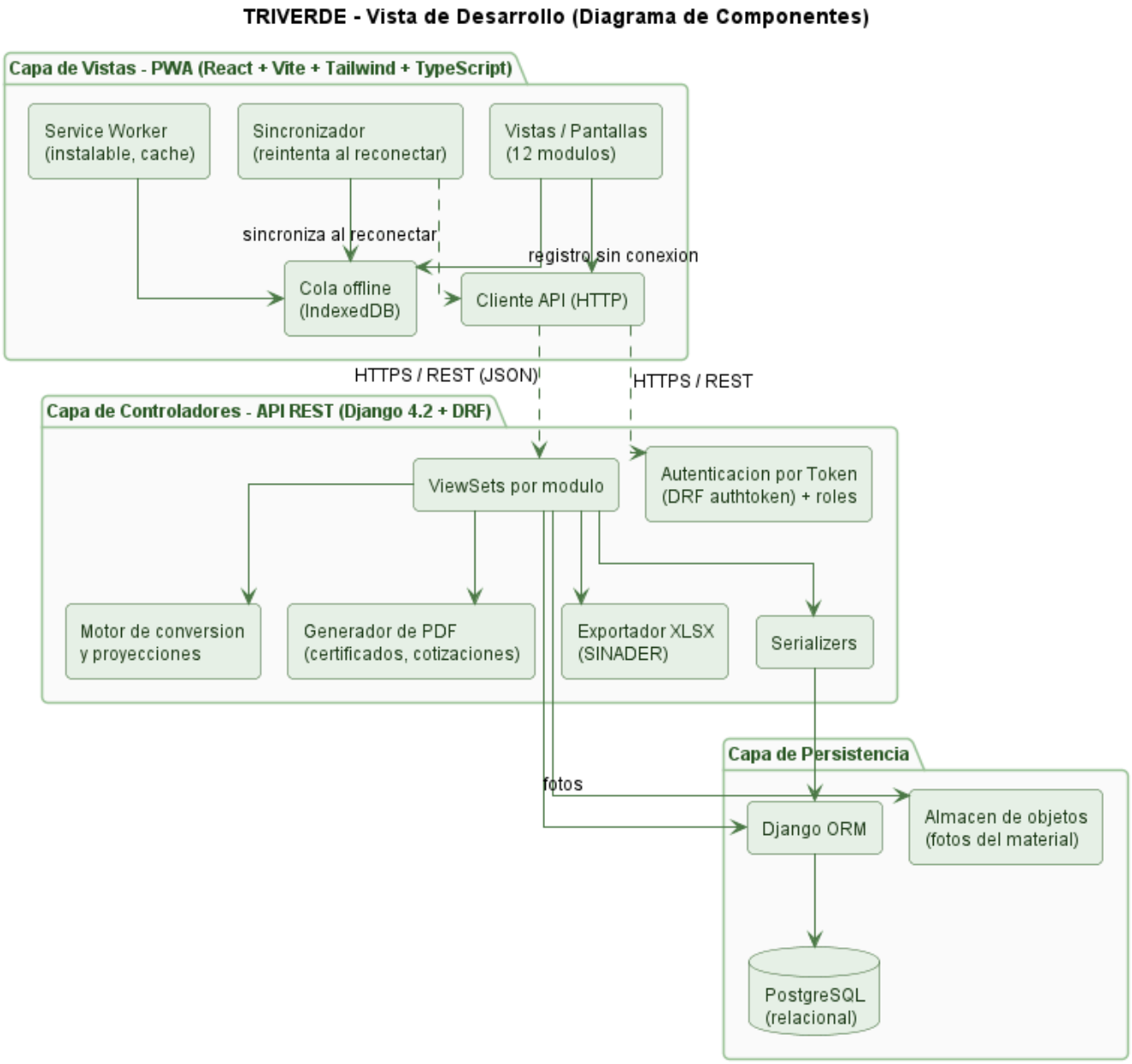


Figura 3.1. Diagrama de componentes (vista de desarrollo)

2. Tecnologías y por qué

Pieza	Tecnología	Por qué se eligió	Frente a qué alternativa
Vistas	**React 18 + TypeScript**	Componentes reutilizables y tipos que detectan errores al escribir	JavaScript puro: los errores aparecen recién al ejecutar
Empaquetado	**Vite 6**	Arranque y recarga casi instantáneos; plugin PWA	Create React App con webpack, más lento
Estilos	**Tailwind CSS 4**	Diseño consistente sin escribir hojas de estilo por pantalla	CSS a mano por componente

Pieza	Tecnología	Por qué se eligió	Frente a qué alternativa
Instalable y offline	**PWA** (`vite-plugin-pwa`) + **IndexedDB**	Se instala en el teléfono como una app, sin tienda, y guarda la cola sin señal	App nativa: dos desarrollos (Android e iOS) y publicación en tiendas
Controladores	**Django 4.2 + Django REST Framework**	"Baterías incluidas": ORM, migraciones, autenticación, admin, validación y viewsets ya resueltos	Flask o Express: casi todo se arma a mano
Persistencia	**PostgreSQL 16**	Base relacional (lo exige la rúbrica), restricciones en la propia base, tipos `jsonb` y `uuid`	SQLite: no apta para producción con varios usuarios
Autenticación	**Token de DRF**	Sin estado en el servidor: encaja con un cliente que pasa tiempo sin conexión	Sesión con cookies: pensada para páginas generadas en el servidor
Exportación	**reportlab** (PDF) y **openpyxl** (Excel)	Generan certificados, cotizaciones, reportes y la planilla SINADER en el servidor	Generar en el navegador: el cálculo quedaría en la vista
Montaje	**Docker Compose**	La base y el servidor se levantan iguales en cualquier máquina con una orden	Instalar Python y PostgreSQL a mano en cada equipo
Pruebas	**unittest de Django** + **coverage** + **Playwright**	Pruebas del servidor contra PostgreSQL real y pruebas de extremo a extremo con navegador	—

3. El servidor (Controladores y Persistencia)

3.1 Una app por módulo

El backend es un proyecto Django (`config/`, la configuración global) con una **app por módulo**, más `common/` con las piezas compartidas:

App	Módulo	Controladores de diseño	Inc
`acceso`	M1 Acceso, usuarios y auditoría	`C_Autenticacion`, `C_Usuarios`, `C_Auditoria`	1
`configuracion`	M2 Configuración	`C_Configuracion`	1-2
`mantenedores`	M3 Mantenedores	`C_Clientes`, `C_Transportistas`, `C_Materiales`, `C_Productos`, `C_Vehiculos`	1
`recepcion`	M4 Recepción y offline	`C_Recepcion`, `C_Sincronizacion`	1

App	Módulo	Controladores de diseño	Inc
`inventario`	M5 Inventario, pilas y procesos	`C_Inventario`, `C_Pilas`, `C_ProcesosPila`	2
`mezcla`	M6 Mezcla y alertas	`C_Mezcla`, `C_Alertas`	2
`proyecciones`	M7 Proyecciones	`C_Proyecciones`	2
`comercial`	M8 Comercial	`C_Cotizaciones`, `C_Ventas`, `C_Cobros`	2
`mantenimiento`	M11 Mantenimiento	`C_Maquinaria`, `C_Mantenimiento`	2
`trazabilidad`	M9 Trazabilidad y ambiental	`C_Trazabilidad`, `C_Ambiental`	3
`reportes`	M10 Reportes y panel	`C_Reportes`	3
`documental`	M12 Gestión documental	`C_Documentos`	3

En total son **26 controladores de diseño** (catálogo `docs/referencia/Vistas y controladores.pdf`). En el código, cada `C_` se implementa como uno o más **ViewSets** de DRF; el docstring de cada ViewSet dice a qué `C_` corresponde.

3.2 Anatomía de una app

Todas las apps repiten la misma estructura. Entendida una, se entienden todas:

Archivo	Contiene	Capa
`models.py`	Las tablas como clases de Python, con sus restricciones	Persistencia
`migrations/`	Los cambios de esquema versionados (`0001_initial.py`, …)	Persistencia
`serializers.py`	Traducción entre objetos y JSON, y validación de entrada	Puente
`services.py`	La regla de negocio de los CU y sus excepciones	Controladores
`views.py`	Los ViewSets: permiso, validación, llamada al servicio, auditoría, respuesta	Controladores
`urls.py`	Registro de los ViewSets en un `router`	Controladores
`signals.py`	Reacciones automáticas a eventos (solo `inventario` y `trazabilidad`)	Controladores
`tests.py`	Pruebas del módulo	Pruebas

3.3 Modelos y ORM

Un **modelo** es una clase que representa una tabla; cada atributo es una columna. El **ORM** de Django traduce entre objetos y filas, así que se consulta sin escribir SQL: `Material.objects.filter(estado="activo")`.

El nombre físico de cada tabla es `app_modelo` (`recepcion_recepcion`, `inventario_pila`, `mezcla_alerta`). Es el nombre que aparece en los diagramas de secuencia (Parte 6).

Las **migraciones** son archivos que describen cómo cambiar la base cuando cambia un modelo. Al levantar el sistema se corre `python manage.py migrate` y la base queda igual al código.

Las relaciones se declaran con `ForeignKey` y su regla al borrar:

- `PROTECT`: impide borrar el padre si tiene hijos (un cliente con recepciones no se borra).
- `CASCADE`: borra los hijos con el padre, solo dentro de un agregado (las líneas de una recepción).
- `SET_NULL`: deja el campo vacío (el autor de un reporte si su usuario desaparece).

3.4 Serializers

El **serializer** traduce entre el objeto y el JSON de la API y **valida** lo que entra. El ejemplo más importante es la recepción (`recepcion/serializers.py`): el cliente envía solo volumen y material, y el servidor calcula el peso y el chip con `derivar_peso_y_chip`. Esos campos son de **solo lectura**: nadie los puede enviar, siempre se calculan en el servidor. El serializer crea la recepción con sus líneas, fotos y multas dentro de una **transacción** (`transaction.atomic`): se guarda todo o nada.

3.5 ViewSets y rutas

Un **ViewSet** agrupa las operaciones de un recurso. Con `ModelViewSet`, listar, ver, crear y editar ya vienen hechos; se agrega solo lo propio del negocio. Las operaciones especiales de un CU se agregan con `@action`, por ejemplo `certificados/generar-descarga/`, `documentos-legales/{id}/renovar/` o `reportes/{id}/exportar/`.

Cada app registra sus ViewSets en un `router` (`urls.py`). `config/urls.py` junta los routers de las 12 apps en uno solo bajo `/api/v1/`. Entrar a `/api/v1/` muestra la lista de todos los recursos.

3.6 Piezas compartidas (`backend/common/`)

Pieza	Archivo	Para qué
Permisos por rol	`permissions.py`	Cuatro clases que todos los ViewSets usan (tabla siguiente)
Auditoría	`auditoria.py`	`registrar_auditoria(usuario, accion, entidad, id, detalle)` escribe en la bitácora (`acceso_bitacoraauditoria`)
Trazas por CU	`trazas.py`	`traza("CU-55", "cotizacion.calculada", ...)` escribe una línea por cada punto de negocio
Capa offline	`models.py`	`SincronizableModel`: agrega `id_local` (UUID) y `estado_sincronizacion` a los objetos que se registran sin señal
Baja lógica y filtros	`views.py`	Mixins: `BajaLogicaMixin` (el DELETE marca `inactivo` en vez de borrar), `FiltroEstadoMixin`, `FiltroVigenteMixin`

Pieza	Archivo	Para qué
Fotos en base64	`fields.py`	`Base64ImageField`: recibe la foto como texto (así viaja también desde la cola offline) y valida que sea imagen real por su firma
Comandos	`management/commands/`	`seed_inicial` (roles, administrador, tarifas), `seed_demo` (datos de demostración de todos los módulos), `seed_e2e`, `revisar_vencimientos_documentales` (CU-90)

Permisos. Todas las clases exigen primero un usuario autenticado, con rol y en estado `activo`:

Clase	Administrador	Operador	Transportista	Dónde se usa
`IsAdministrador`	Todo	No	No	Usuarios, auditoría, costos, trazabilidad, reportes, documentos, cotizaciones
`IsOperadorOAdministrador`	Todo	Todo	No	Sincronización, procesos de pila, historial de parámetros
`EsAdministradorOOperadorLectura`	Todo	Solo lectura	No	Mantenedores, parámetros, tarifas, recetas
`EsOperadorOAdministradorOTransportista`	Todo	Todo	Solo crear y leer	Recepciones. Además el ViewSet filtra para que el transportista vea solo las suyas

Trazas. Sirven para comprobar en las pruebas de extremo a extremo que un cálculo lo hizo el servidor y no la pantalla. Si una vista copiara la fórmula de la cotización en JavaScript, la pantalla mostraría el número correcto pero se rompería el patrón por capas. La traza `CU-55 cotizacion.calculada` en el registro del servidor demuestra que el cálculo pasó por donde debía.

3.7 Autenticación por token

1. La vista de login envía usuario y contraseña a `POST /api/v1/auth/login/` (`acceso/views.py`, `LoginView`).
2. El servidor verifica la contraseña (guardada con hash, nunca en texto) y que el usuario esté activo, y responde con un **token** y los datos del usuario, incluido su rol.
3. Las Vistas guardan el token y lo envían en cada petición en la cabecera `Authorization: Token <clave>` (`frontend/src/lib/api.ts`).

4. Cada ViewSet revisa el token y el rol con sus clases de permiso. Sin token: 401. Con token pero sin permiso: 403.
5. `POST /api/v1/auth/logout/` invalida el token.

El token es **sin estado**: el servidor no guarda una sesión abierta, lo que conviene a una app que pasa ratos sin conexión y reintenta al volver.

4. Las Vistas (PWA)

4.1 Estructura

```
frontend/src/
  main.tsx           punto de entrada
  App.tsx            mapa de rutas: qué dirección muestra qué pantalla (35 rutas)
  components/ui/     piezas reutilizables (botones, tablas, tarjetas, modales)
  components/layout/ armazón: menú lateral y encabezado
  pages/             pantallas por área: recepcion, mantenedores, configuracion, operacion,
                     comercial, mantenimiento, trazabilidad, gestion
  lib/api.ts         cliente HTTP único (axios) que agrega el token
  lib/auth.tsx       contexto de sesión: usuario actual, login, logout
  lib/navegacion.ts  menú según el rol
  lib/types.ts       tipos TypeScript, espejo de los serializers
  lib/db.ts          acceso a IndexedDB
  lib/sync.ts        cola offline: encolar y sincronizar
  lib/useOnline.ts   detecta si hay conexión
```

4.2 Control de acceso en dos capas

`RutaProtegida` impide entrar sin sesión y `navegacion.ts` muestra a cada rol solo sus módulos. Eso es **comodidad**. La **seguridad** está en el servidor: aunque alguien escriba la dirección a mano, la API responde 403 si su rol no tiene permiso. La regla de verdad siempre está en el servidor.

4.3 Una pantalla de punta a punta

`pages/recepcion/RegistroRecepcion.tsx`: al abrir pide los catálogos (clientes, materiales, vehículos, transportistas); al enviar arma el JSON **sin peso** y llama a `api.post("/recepciones/", payload)`; la respuesta ya trae el peso calculado y la pantalla lo muestra. Si no hay conexión, o la red falla, en vez de perder el dato lo encola con `encolarRecepcion(payload)`. Si el servidor responde con un error de validación, lo muestra y **no** encola, porque reintentarlo no lo arreglaría.

5. Operación sin conexión

5.1 La cola

La cola vive en **IndexedDB**, la base de datos que trae todo navegador, y guarda objetos completos, incluida la foto en base64. Al encolar se genera un `id_local` con `crypto.randomUUID()` y la operación queda `pendiente`. `localStorage` no sirve para esto: solo guarda texto corto.

5.2 La sincronización idempotente

Cuando vuelve la señal (`useOnline.ts` escucha los eventos `online` y `offline` del navegador), `sincronizar()` envía la cola completa a `POST /api/v1/sincronizacion/` (`recepcion/views.py`, `SincronizacionView`). El servidor:

1. Ordena las operaciones por `capturado_en` (el momento en que se registraron en el dispositivo).
2. Procesa cada una en su propia transacción, según su tipo: `recepcion` o `proceso_pila` (los procesos de pila se delegan a `inventario/sincronizacion.py`, para que el Módulo 4 no dependa del 5).
3. **Busca por `id_local` antes de crear.** Si ya existe y está sincronizada, responde "sincronizada" sin crear otra. Si no existe, la crea con ese mismo `id_local`.
4. Si los datos no pasan la validación, responde "rechazada" con el motivo, y la operación queda **en conflicto** en la bandeja `V_ColaSincronizacion` para revisión, sin descartarse.

Esto es **idempotencia**: enviar el mismo lote dos veces da el mismo resultado que enviarlo una vez. Si la señal se corta después de guardar pero antes de recibir la confirmación, el reintento no duplica el camión. En la base, el `id_local` tiene restricción única, así que tampoco podría duplicarse por otro camino.

La PWA encola hoy las recepciones. El servidor además acepta procesos de pila (`proceso_pila`), porque `ProcesoPila` también hereda de `SincronizableModel` y se registra en el patio.

6. El recorrido completo

Registrar una recepción con conexión, archivo por archivo:

1. **Vista** `RegistroRecepcion.tsx` arma el JSON (cliente, vehículo, líneas con volumen) y llama a `api.post`.
2. **Cliente HTTP** `lib/api.ts` agrega el token y envía a `/api/v1/recepciones/`.
3. **Router** `recepcion/urls.py` dirige al `RecepcionViewSet` (`C_Recepcion`).
4. **Permiso** `EsOperadorOAdministradorOTransportista` revisa rol y estado.
5. **Serializer** valida, deriva peso y chip (`volumen × densidad`, `volumen ÷ factor`) y crea todo en una transacción.
6. **ORM** guarda en `recepcion_recepcion`, `recepcion_detallerecepcion`, `recepcion_fotorecepcion`.
7. **Señales**: el inventario recibe el material en etapa `por triturar` (CU-44) y se calcula el CO_2 del camión (CU-69) si la recepción queda `recibida` y hay factor configurado.
8. **Respuesta** JSON con el peso derivado; la pantalla muestra, por ejemplo, "20 m^3 → 2.300 kg".

Sin conexión, entre el paso 1 y el 2 la vista desvía el JSON a la cola, y al reconectar la sincronización aplica los pasos 3 a 7 de forma idempotente. El resultado es el mismo.

Cualquier otra funcionalidad sigue el mismo camino, casi siempre más simple.

7. Instalación y despliegue

7.1 Cómo se levanta

`backend/docker-compose.yml` define dos servicios:

- `db`: PostgreSQL 16 con un volumen persistente.
- `web`: el servidor Django. Al arrancar ejecuta `migrate` (crea o actualiza tablas), `seed_inicial` (roles, administrador, tarifas), `collectstatic` y levanta **Gunicorn** en el puerto 8000.

Paso	Comando (desde la carpeta indicada)
Levantar base y servidor	`backend/`: `docker compose up -d`
Cargar datos de demostración	`backend/`: `docker compose exec -T web python manage.py seed_demo`
Correr las pruebas	`backend/`: `docker compose exec -T web python manage.py test`
Levantar las Vistas	`frontend/`: `npm ci` (primera vez) y `npm run dev -- --host`
Revisar vencimientos (CU-90)	`backend/`: `docker compose exec -T web python manage.py revisar_vencimientos_documentales`

Sin `--host`, Vite puede escuchar solo en IPv6 y el navegador no conecta. En Windows, tras editar el backend hay que reiniciar el contenedor (`docker compose restart web`) porque no detecta los cambios solo. El detalle está en el documento de instalación de cada incremento.

7.2 Arquitectura de producción

El diagrama de despliegue (Figura 3.2) muestra la arquitectura de producción: en un servidor con Docker, la API (Django con Gunicorn), las Vistas compiladas servidas por Nginx y PostgreSQL; el dispositivo del usuario accede por HTTPS. HTTPS es obligatorio en producción porque el navegador solo permite usar la cámara en sitios seguros (RNF-11).

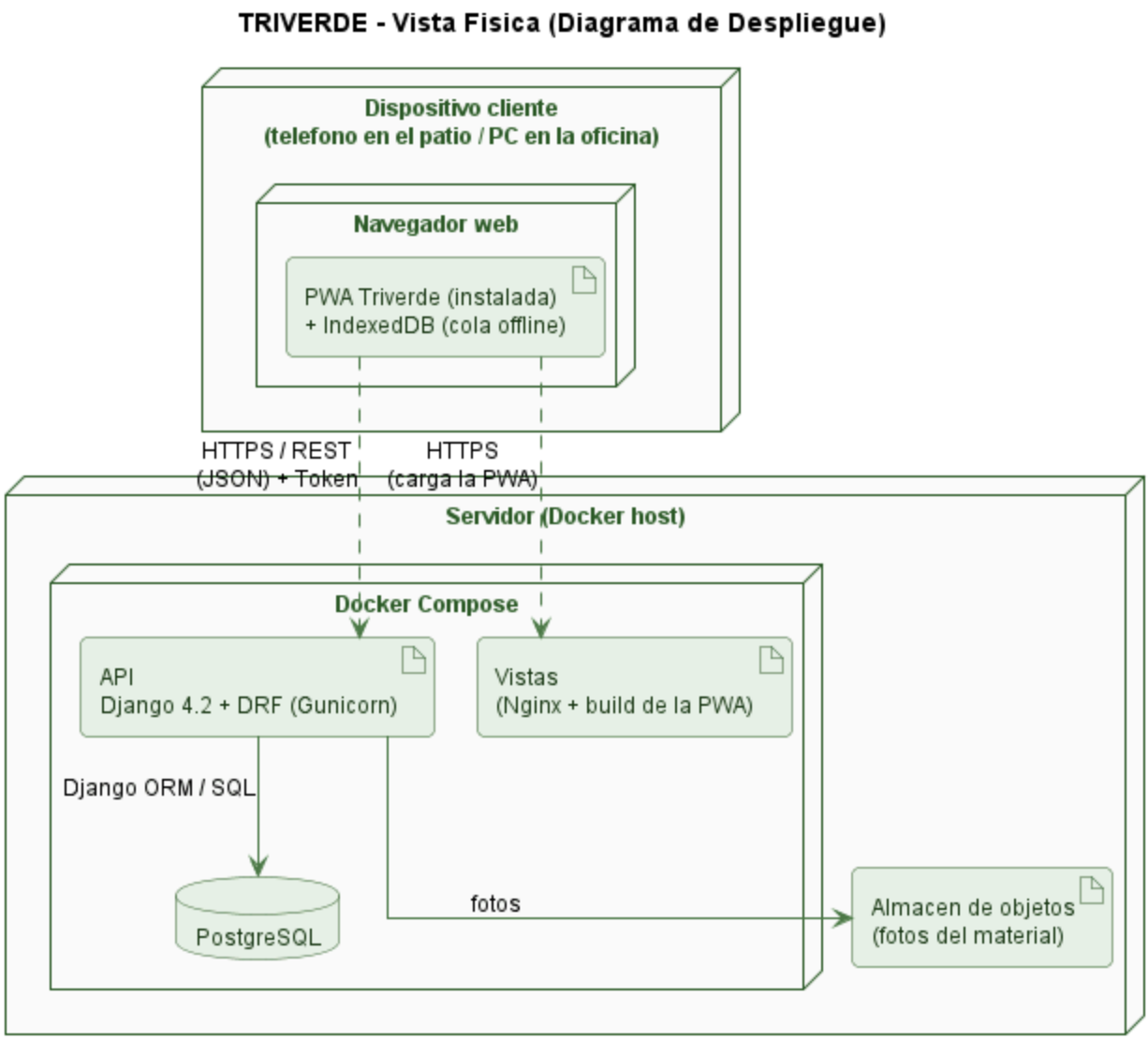


Figura 3.2. Diagrama de despliegue (vista física)

En desarrollo y en las demostraciones, las Vistas corren con el servidor de Vite. La puesta en producción en un servidor propio está planificada para cuando la planta opere con su resolución sanitaria; incluye programar una vez al día el comando de vencimientos documentales. Como todo está en Docker Compose, cambiar de máquina no obliga a reconfigurar nada (RNF-09).

8. Glosario

- **API**: conjunto de direcciones por las que el servidor ofrece datos y operaciones. Aquí, bajo `/api/v1/`.
- **Baja lógica**: marcar un registro como `inactivo` en vez de borrarlo, para no romper la historia.
- **Endpoint**: una dirección concreta de la API, como `/api/v1/recepciones/`.
- **Idempotente**: que repetir la operación da el mismo resultado que hacerla una vez.
- **IndexedDB**: base de datos del navegador; guarda la cola offline.
- **JSON**: formato de texto con el que se intercambian datos Vistas y Controladores.
- **Migración**: archivo que actualiza la base cuando cambia un modelo.

- **ORM**: traductor entre clases de Python y tablas, sin escribir SQL.
- **PWA**: aplicación web instalable que puede funcionar sin conexión.
- **Serializer**: traductor entre objetos y JSON; también valida.
- **Señal (signal)**: función que Django ejecuta sola cuando ocurre un evento, como guardar un objeto.
- **Service worker**: script del navegador que hace la app instalable y guarda en caché sus archivos.
- **Token**: credencial que las Vistas envían en cada petición tras iniciar sesión.
- **ViewSet**: clase de DRF que agrupa las operaciones de un recurso; implementa un `C_`.

9. Preguntas de repaso

¿Qué arquitectura tiene el sistema? Cliente-servidor en tres capas: Vistas (PWA en React), Controladores (API REST en Django REST Framework) y Persistencia (PostgreSQL). Se comunican solo por JSON.

¿Dónde está la lógica de negocio? En el servidor, en los servicios de cada app. Las Vistas no calculan nada.

¿Cómo demuestran que la pantalla no calcula? Con las trazas por CU: las pruebas de extremo a extremo verifican en el registro del servidor que el cálculo ocurrió allí.

¿Por qué Django y no otro framework? Trae resuelto el ORM, las migraciones, la autenticación y la validación, lo que permite concentrarse en las reglas del negocio.

¿Por qué una PWA y no una app nativa? Se instala en cualquier teléfono o computador desde el navegador, sin tiendas, con un solo desarrollo, y puede funcionar sin conexión.

¿Cómo funciona sin señal? Guarda la operación en IndexedDB con un identificador único generado en el dispositivo. Al volver la señal la envía al endpoint de sincronización, que la crea solo si ese identificador no existe.

¿Qué pasa si se envía dos veces la misma recepción? Nada: el servidor la reconoce por su `id_local` y responde que ya está sincronizada. Además el `id_local` es único en la base.

¿Qué pasa si la recepción encolada tiene un error? El servidor la rechaza con el motivo y queda en conflicto en la bandeja de sincronización para revisarla, sin perderse.

¿Cómo se controla quién puede hacer qué? Con permisos por rol en cada controlador del servidor. El menú de las Vistas solo oculta lo que no corresponde; la prohibición real está en la API.

¿Qué es un ViewSet y qué relación tiene con los `C_`? Es la clase de DRF que agrupa las operaciones de un recurso. Cada controlador de diseño `C_` se implementa como uno o más ViewSets.

¿Por qué token y no sesión? El token no obliga al servidor a recordar sesiones, lo que encaja con un cliente que pasa tiempo sin conexión.

¿Para qué sirve Docker? Para levantar la base y el servidor iguales en cualquier máquina con una orden, sin instalar ni configurar Python y PostgreSQL a mano.

¿Por qué hace falta HTTPS? Porque el navegador solo permite usar la cámara (la foto del material) en sitios seguros.

¿Qué es una migración? Un archivo que dice cómo cambiar la base cuando cambia un modelo, para que la base y el código coincidan.